FINAL-YEAR ENGINEERING PROJECT | IMPLEMENTED PROTOTYPE

CareerNavigator AI

Live Job Market Analytics, Career Recommendation, Skill-Gap Analysis and Personalized Learning Roadmap System

REVIEW 1 SYNOPSIS

Implementation-aligned synopsis describing the current prototype, methods, constraints, and next validation steps.

Implementation status as at 13 October 2026

Student name: _____ Roll number: _____

Department / institution: _____

Project guide: _____ Academic year: _____

ABSTRACT

CareerNavigator AI is a web-based prototype that relates a student's self-reported skills and interests to an observed sample of job-posting requirements. SerpApi Google Jobs provides bounded user-initiated retrieval. A FastAPI backend maps results into a common job shape, cleans and deduplicates records, extracts skills from an editable dictionary, and stores raw snapshots and normalized jobs in a relational database. The React interface presents sample-level market analytics and supports explainable role matching, skill-gap prioritization, and weekly learning roadmaps. Matching is weighted content-based overlap with curated role requirements, not a trained employment prediction model. Metrics describe only collected records; missing fields and historic trends are not invented.

Scope statement
This synopsis distinguishes implemented behavior from limitations and proposed evaluation. It does not claim hiring outcomes, model accuracy, representative market coverage, or educational impact.

1. Problem, motivation, aim, and objectives

Problem statement

Students need a transparent way to relate their current skills to role-specific requirements and observable job-posting signals, then turn skill gaps into practical learning steps. Job listings are distributed across sources and cannot be compared easily by title alone.

Aim

Design and implement a web application that collects permitted job-search results, summarizes skills in the collected sample, compares student profiles with curated career roles, identifies skill gaps, and generates a personalized weekly learning roadmap.

Objectives

1. Retrieve Google Jobs results through a server-side SerpApi integration with bounded pagination, timeout, and error handling.

2. Normalize and deduplicate postings, preserve posted_date separately from collected_at, extract skills from an editable dictionary, and report data quality.

3. Persist raw snapshots, cleaned jobs, canonical skills, and relations in PostgreSQL or MySQL.

4. Display filtered sample counts, skill-demand percentages, source and date coverage metadata where available.

5. Explain weighted role matching and classify skills as possessed, partial, or missing.

6. Prioritize gaps using role importance and available observed job demand; build learning milestones with practice and curated resources.

7. Protect API credentials, minimize profile data retention, and test behavior with automated checks.

2. Scope, feasibility, and implemented status

Module	Status and limitation
Live collection	Implemented SerpApi Google Jobs adapter; requires private key and uses account quota. One bounded integration smoke query was verified.
Cleaning and extraction	Implemented title/location normalization, duplicate handling, skill dictionary, and quality report. Dictionary matching can miss context.
Storage	SQLAlchemy models/initialization support PostgreSQL and MySQL; tests use SQLite. Market persistence requires database setup.
Analytics	Filtered saved-sample counts, skills, locations, roles, source and date metadata. No unsupported salary/experience/trend claims.
Career match	Four curated role profiles and transparent weighted rule-based coverage. Not a trained ML or hiring-probability model.
Gap and roadmap	Possessed/partial/missing gaps, market-aware priority when saved data exists, weekly plans and catalog resource links.
Privacy	Submitted student profile is analyzed but not saved by current recommendation flow. Roadmap persistence is separate; progress checkboxes are browser-local.
Formal outcome evaluation	Not performed. No hiring outcome, accuracy, fairness, or educational-impact metrics are claimed.

Technology actually used

Python, FastAPI, Pydantic, httpx, SQLAlchemy, PostgreSQL/MySQL, React, Vite, JavaScript, Plotly visual components, JSON reference catalogs, pytest, and Oxlint. The current implementation does not claim Pandas/NumPy analytics or a scikit-learn trained recommender; matching is implemented as explicit Python rules.

3. Architecture and data flow

Component	Responsibility
React + Vite	Student forms, search/results, dashboard, career guidance views, responsive layout, loading and error feedback.
FastAPI	Validated REST endpoints, provider orchestration, cleaning, analytics, career logic, OpenAPI docs.
SerpApi collector	Google Jobs requests, India/language defaults, bounded pagination, timeout/error translation, common job field mapping.
Processing services	Normalize fields, deduplicate, extract canonical skills, compute quality and market summaries.
SQL database	Persist raw snapshots, jobs, skills, role/resource reference data, and generated roadmaps.
JSON catalogs	Editable skills/aliases, role descriptions and requirement weights, learning resources.

Data flow

1. Student submits job query and location. 2. Backend reads private key and fetches bounded results. 3. Frontend previews returned postings without saving. 4. Student selects Clean & save; backend validates, processes, stores raw snapshots, and upserts normalized jobs/skills. 5. Dashboard filters query the saved database. 6. Career endpoints compare profile inputs with curated role requirements and return recommendations, gap groups, or a learning plan.

> **Trust boundary**
> Provider credentials stay in backend .env and must never be embedded in React, committed to Git, or shown in screenshots.

4. Database design and core methods

Entities	Description
raw_job_postings; jobs	Source payload snapshots and normalized/upserted job records, including provider identity, dates, collection time, and source URL.
skills; job_skills	Canonical skill names/categories/aliases and job-skill many-to-many links.
career_roles; career_role_skills	Role catalog and weighted requirements/minimum skill levels.
student_profiles; student_skills	Relational schema is defined; current request-based profile analysis does not persist profile inputs.
learning_resources	Skill-linked URLs, provider/type, and verification metadata.
learning_roadmaps; roadmap_milestones	Generated roadmap, sequential weeks, goals, practice, projects, resources, completion state.

Skill demand (%) = 100 x unique selected jobs mentioning a skill / all unique selected jobs

Recommendation match (%) = 100 x matched role-requirement weight / total role-requirement weight

A reported skill below the encoded role minimum contributes 50% of its requirement weight. Gaps are grouped as possessed, partial, or missing. Partial/missing priority uses role weight with observed skill demand when available; without market data, role importance alone is used. Roadmap milestones are created from prioritized gaps and draw only from curated resource URLs.

5. API, interface, and verification

Endpoint group	Purpose
GET /api/health	Backend process health.
GET /api/jobs/search; POST /api/jobs/process	Bounded live search, then explicit processing/persistence.
POST /api/jobs/clean	Normalize a supplied batch and report quality without persisting.
GET /api/market/summary, /skills, /jobs	Filtered saved-sample analytics and posting retrieval.
GET /api/careers/roles	List curated role catalog.
POST /api/careers/recommendations, /skill-gaps, /roadmaps	Role match, prioritized gap analysis, and weekly roadmap.

Frontend screens

Overview; Job Market live preview/clean-save and saved-sample filters; Career Match; Skill Gaps; Learning Roadmap; Sources & Project.

Recorded verification

48 backend tests passed at the recorded verification point. Frontend lint and production build passed; build reports a non-blocking large Plotly chunk warning. One bounded live SerpApi query returned 10 postings in one page. The smoke request is not a representative sample or a model evaluation.

6. Limitations, future work, and references

Limitations

The provider’s access/quota and result coverage can change. Job descriptions may omit data; relative posting dates are not converted into exact dates. Salary/currency and structured experience data are not currently reliable from the active mapping. A single collection cannot establish a historical trend. Skill extraction is dictionary-based; role definitions and weights are curated assumptions. Findings apply only to the collected sample. Match scores are not hiring probabilities. The system does not claim a trained ML model, model accuracy, fairness validation, or employment outcomes.

Future work

Build comparable scheduled snapshots; annotate a sample for skill-extraction evaluation; review role weights/resources with advisors; conduct accessibility and usability checks; and define data, protocol, and metrics before any statistical or user-impact evaluation.

References

[1] SerpApi Google Jobs API: https://serpapi.com/google-jobs-api. [2] FastAPI documentation: https://fastapi.tiangolo.com. [3] React Learn: https://react.dev/learn. [4] Vite guide: https://vite.dev/guide. [5] Plotly: https://plotly.com. [6] PostgreSQL documentation: https://www.postgresql.org/docs/. Verify provider quotas/terms and linked resources before submission.

Before submission
Fill in student, institution, guide, and academic-year fields. Confirm that the department accepts this Review 1 format and verify the stated implementation against the demo version.